

Elden Ring Agentic RAG Guide

Entrega MVP — Guía conversacional agéntica con RAG, memoria persistente y despliegue en producción

Autora:	Estefanía Alonzo
Proyecto GCP:	ah-estefania-alozno
Servicio activo:	https://elden-ring-agent-704637212685.us-central1.run.app
Fecha:	5 de septiembre de 2026
Documento fuente:	PRD_Elden_Ring_Agentic_RAG_MVP.md

1. Problema y objetivo

El proyecto Medallion previo (`practica_uno`) produjo un Data Product terminado: una capa Gold en BigQuery con ~1,208 entidades de Elden Ring (armaduras, armas, bosses, incantations, cenizas de guerra y NPCs) en `semantic_documents` , más una primera estrategia de embeddings (`entity_embeddings` , E5 local, 384-dim). Ese Data Product no tenía, sin embargo, ninguna forma de que un jugador conversara con él: no había agente, no había memoria por usuario, no había autenticación ni un canal de recomendación personalizada.

- El objetivo de este MVP es construir esa capa faltante — un agente conversacional agéntico (Google ADK) que:
- responda preguntas de lore/conocimiento **grounded exclusivamente** en el corpus recuperado por RAG, nunca en el conocimiento preentrenado del LLM;
 - recomiende equipo (arma/armadura/incantation/ceniza) personalizado al perfil del jugador cuando la petición lo amerita, devolviendo hasta 3 opciones razonadas;
 - recuerde preferencias y stats del jugador de forma persistente entre sesiones, sin onboarding explícito — las detecta de la conversación natural;
 - aisle completamente la memoria y el historial entre usuarios distintos;
 - corra en producción real (Cloud Run), no solo en un notebook o entorno local.

No es un ejercicio de "wrapear un LLM" — el principio no negociable del proyecto (P-01, grounding estricto) es que ningún hecho sobre Elden Ring puede salir del modelo sin pasar primero por `search_elden_ring_knowledge` contra el corpus real.

2. Reutilización del proyecto Medallion

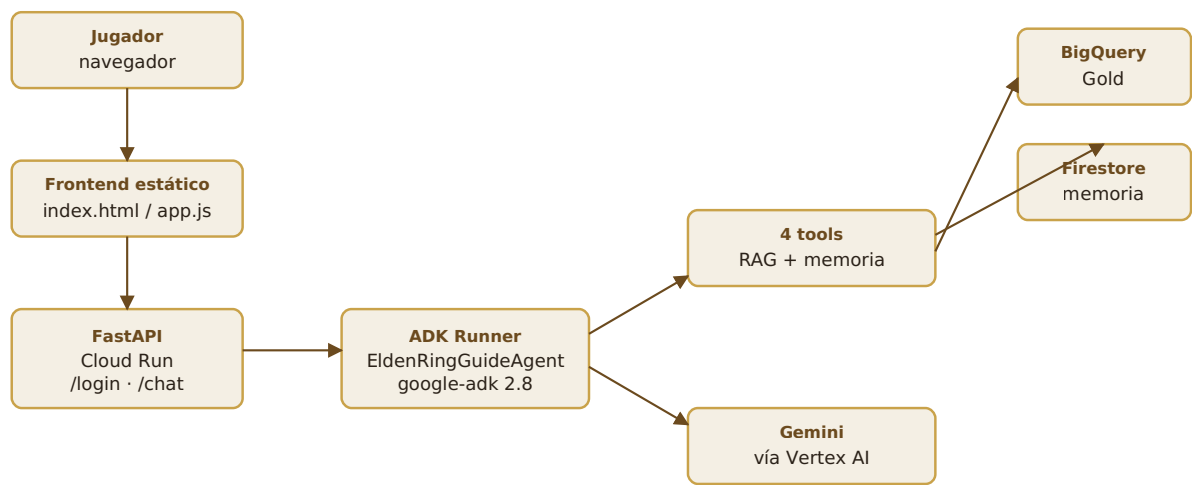
Bronze → Silver → Gold → Agentic RAG (este proyecto)

Este proyecto **no reconstruye** nada de las capas Bronze/Silver/Gold del proyecto Medallion — las consume como Data Product terminado, de solo lectura:

Tabla/artefacto	Origen	Uso en este proyecto
<code>semantic_documents</code>	Gold, capa de texto	Contrato de texto (<code>searchable_text</code>) — solo lectura, sin cambios.
<code>entity_embeddings</code> (E5, 384-dim)	Gold, estrategia legado	Intocable. No se lee ni se usa en este RAG — es evidencia de un proyecto anterior.
<code>entity_embeddings_vertex</code> (Vertex, 768-dim)	Gold, extensión paralela documentada en <code>VERTEX_RAG_HANDOFF.md</code>	Tabla que consume la tool <code>search_elden_ring_knowledge</code> vía <code>VECTOR_SEARCH</code> . 1,208 filas, 100% de cobertura frente a <code>semantic_documents</code> .

El handoff (`VERTEX_RAG_HANDOFF.md` , copiado a este repo) documenta el contrato exacto: modelo `gemini-embedding-001` , dimensión 768, `task_type` asimétrico (`RETRIEVAL_DOCUMENT` para el corpus, `RETRIEVAL_QUERY` para las preguntas del usuario) y el patrón de `ML.GENERATE_EMBEDDING` + `VECTOR_SEARCH` replicado en `backend/repositories/bigquery_repository.py` .

3. Arquitectura



Jugador → Frontend estático (mismo origen) → FastAPI en Cloud Run → ADK Runner → EldenRingGuideAgent → 4 tools → BigQuery (RAG) / Firestore (memoria) / Gemini vía Vertex AI.

Desviación frente al diagrama original del PRD: el PRD proponía Apps Script embebido en Google Sites como frontend. Se implementó y se publicó, pero el navegador real nunca pudo renderizar el Web App pese a que el servidor ejecutaba `doGet()` correctamente en el 100% de los intentos (ver sección 10, Decisiones y trade-offs). Se reemplazó por un frontend estático servido por el mismo FastAPI — mismo origen que la API, sin CORS, sin Google Sites. La URL de Cloud Run es la aplicación completa.

4. RAG

Corpus

1,208 documentos en `entity_embeddings_vertex` (armor 563, weapon 286, boss 105, incantation 100, ash 94, npc 60), cobertura 100% frente a `semantic_documents`.

Embeddings

Campo	Valor
Modelo	<code>gemini-embedding-001</code> (Vertex AI, vía modelo remoto de BigQuery ML)
Dimensión	768
Task type documentos	<code>RETRIEVAL_DOCUMENT</code>
Task type queries	<code>RETRIEVAL_QUERY</code>

Query embedding

Cada pregunta del usuario se vectoriza en el momento con `ML.GENERATE_EMBEDDING`, usando el mismo modelo remoto (`elden_ring_embedding_model`) y la misma dimensión que los documentos — el patrón asimétrico `RETRIEVAL_QUERY/RETRIEVAL_DOCUMENT` es obligatorio para que la comparación de distancia sea comparable (mezclarlos degrada la recuperación en silencio, sin error).

Vector Search

`VECTOR_SEARCH` sobre `entity_embeddings_vertex`, sin índice ANN — con ~1,208 documentos el escaneo exacto (`distance_type => 'COSINE'`) es suficiente y evita complejidad de infraestructura prematura (TODO-07). Filtro opcional por `entity_types` antes de la búsqueda.

Grounding y fuentes

El agente **no tiene un check de backend que bloquee una respuesta sin evidencia** — el grounding aquí es una **mitigación de prompt, no una garantía dura a nivel de código**. El system instruction (`backend/agent/instructions.py`) le exige a Gemini respaldar todo hecho en resultados de `search_elden_ring_knowledge` y rechazar preguntas fuera de corpus con una frase fija, pero `backend/tools/rag.py` y el endpoint `/chat` no validan después del hecho que la respuesta final cite realmente las entidades recuperadas, ni impiden una respuesta si el modelo decide no llamar la tool. Es una política declarada, verificada empíricamente con 13 casos reales en la evaluación (sección 9) — no una validación técnica en tiempo de ejecución.

	Fuera de corpus (rechazo)	Dentro de corpus (con fuentes)
Pregunta	"What is the capital of France?"	"Given what you know about me, what weapon should I use?" (tras declarar preferencia por katanas)
Respuesta	"I don't have enough information in my knowledge base to answer that with confidence."	Recomienda Nagakiba, Meteoric Ore Blade y Uchigatana con razones ligadas al perfil.
Fuentes	(ninguna — correcto, no hay retrieval que citar)	<code>weapon · Nagakiba</code> , <code>weapon · Meteoric Ore Blade</code> , <code>weapon · Uchigatana</code>

Transcripts completos y reales (no fabricados): casos `out-of-kb-real-world` y `recommendation-memory` en `evaluation/transcripts.json`.

5. Sistema agéntico

ADK

`google-adk` 2.8.0 — una API basada en `Agent / Runner / Event`, distinta de tutoriales más antiguos del framework. `Agent.tools` acepta funciones Python planas directamente (ADK las envuelve internamente), lo que permite registrar las tools como closures construidas en tiempo de request.

El agente: `EldenRingGuideAgent`

Un único agente con exactamente 4 tools:

Tool	Función
<code>search_elden_ring_knowledge</code>	RAG — recuperación semántica sobre BigQuery.
<code>get_player_profile</code>	Lee preferencias/stats del jugador autenticado.
<code>update_player_memory</code>	Guarda/corrigie preferencias/stats (merge parcial).
<code>forget_player_memory</code>	Borra campos específicos del perfil (dotted path).

El `user_id` se inyecta por closure al construir el agente (`build_agent(user_id, ...)`) — nunca es un parámetro que el LLM pueda rellenar. Esto es no negociable (P-03 / PRD §12.4): ninguna tool acepta ni puede aceptar un `user_id` arbitrario del modelo o del payload del chat.

¿Por qué un solo agente y no multiagente?

El alcance funcional (RAG + memoria + recomendación) no requiere orquestación entre agentes especializados — un único agente con 4 tools cubre el flujo completo sin la complejidad adicional de handoffs, estado

compartido entre agentes, o enrutamiento de intención (P-04, complejidad mínima). Introducir sub-agentes sin una necesidad funcional nueva sería sobre-ingeniería para este alcance.

6. Memoria

Tres conceptos deliberadamente separados, que no deben mezclarse (P-03):

Concepto	Dónde vive	Sobrevive a...
Contexto de sesión	Sesión ADK (en memoria del proceso)	Nada — se reinicia en cada login, evita contexto infinito y comportamiento impredecible.
Memoria persistente del jugador	Firestore, <code>player_profiles/{user_id}</code>	Logins, redeploy, todo — sobrevive indefinidamente hasta que se olvide explícitamente.
Transcript de conversación	Firestore, <code>chat_sessions/{session_id}/messages</code>	Logins — navegable desde un panel dedicado (TODO-01, ver "Actualización post-MVP" abajo), pero no se re-inyecta automáticamente como contexto activo del agente al reabrir una conversación.

El aprendizaje es progresivo y sin onboarding: el agente detecta preferencias/stats mencionados naturalmente en la conversación y llama `update_player_memory` por su cuenta. Corrección ("ya no juego X, ahora Y") reemplaza el valor, no lo acumula. Olvido explícito borra solo los campos pedidos. Todo esto se validó en vivo contra Firestore real (Fase 3 y en los casos `memory-correction` / `memory-forget` de la evaluación).

Bug real encontrado y corregido: el cliente Python de Firestore no interpreta claves con punto (`"stats.level"`) como rutas anidadas dentro de `set(..., merge=True)` — las guarda como nombres de campo literales. El merge parcial correcto usa `update()` con esas mismas claves (que sí resuelve la ruta), asegurando primero que el documento exista.

Actualización post-MVP: historial navegable (TODO-01)

Tras cerrar el MVP se implementó el selector de historial que quedaba explícitamente fuera de alcance (TODO-01, sección 12). Tres endpoints nuevos, todos filtrados por el `user_id` del JWT igual que `/chat : GET / sessions` (lista las conversaciones propias, más recientes primero, con preview del primer mensaje), `GET / sessions/{id}/messages` (transcript completo de una conversación propia) y `POST /sessions` (inicia una conversación nueva sin cerrar sesión). En el frontend, un panel "Mis conversaciones" en la pantalla de Chat permite reabrir y continuar cualquier conversación anterior. Junto con esto se agregó un indicador de progreso ("Pensando..." / "Buscando información...") mientras `POST /chat` está en curso, para que la UI nunca parezca congelada durante la vuelta RAG + LLM.

Bug real y corregido — la memoria no se guardaba de forma confiable: en una conversación real el usuario declaró una preferencia de playstyle y, en una sesión posterior, el agente no la recordaba; Firestore confirmó `player_profiles` vacío pese a la declaración explícita. Causa: el modelo no siempre llamaba `update_player_memory` cuando la preferencia llegaba como respuesta a una pregunta del propio agente dentro de un flujo de recomendación. Se corrigió con una regla explícita atada a ese paso del prompt, y se verificó reproduciendo la conversación original. **Sigue siendo probabilístico:** la extracción de memoria depende de que el modelo decida llamar la tool — no hay validación de backend que compruebe la persistencia después del hecho. El caso `recommendation-memory` de la evaluación (sección 9, 3 turnos: declarar preferencia → recomendación que la usa) queda como prueba de regresión para este escenario exacto.

Bug real y corregido — conversaciones vacías: la primera versión del historial creaba el documento `chat_sessions` en cada login y cada "Nueva conversación", aun sin mensajes; el panel se llenaba de entradas vacías. Se corrigió con creación perezosa (el documento se crea recién en el primer mensaje real, vía `POST /chat`) y se limpiaron, una sola vez, 41 de 53 documentos vacíos ya existentes en Firestore.

7. Autenticación

Solo `username` + `password` — sin email, verificación, recuperación de contraseña ni MFA (deuda técnica aceptada explícitamente, ver Limitaciones).

Aspecto	Implementación
Hashing	PBKDF2-HMAC-SHA256, salt aleatorio de 16 bytes, 260,000 iteraciones. Nunca texto plano.
Token	JWT (HS256), <code>user_id</code> (UUID interno) en el payload, TTL configurable (480 min por defecto).
Aislamiento	<code>user_id</code> sale exclusivamente del token decodificado por la dependencia de FastAPI — nunca del payload del chat ni elegido por el LLM.

Decisión posterior al PRD original: se cerró el registro público (`POST /register` ya no existe). El frontend es de acceso anónimo, y la dueña del proyecto no quería que cualquier visitante creara una cuenta y consumiera tokens de Gemini a su cargo. Los usuarios son un roster fijo (3 cuentas) creado localmente vía `scripts/create_user.py`, nunca por HTTP.

8. Despliegue

Componente	Estado
Cloud Run	activo servicio <code>elden-ring-agent</code> , <code>us-central1</code> , build desde Dockerfile vía <code>gcloud run deploy --source .</code>
Frontend	activo estático, mismo contenedor/origen que la API (ver sección 3 y 10).
Google Sites / Apps Script	no usado reemplazado por el punto anterior — ver Decisiones y trade-offs.
Firestore	activo modo Native, <code>us-central1</code> , colecciones <code>users</code> , <code>player_profiles</code> , <code>chat_sessions</code> (+ subcolección <code>messages</code>).
BigQuery	activo lectura de <code>entity_embeddings_vertex</code> , sin escritura desde este backend.
Secret Manager	activo <code>JWT_SECRET</code> real (fuerte, generado con <code>secrets.token_urlsafely</code>), nunca en variables de entorno de texto plano ni en el repo.
Service account (runtime)	privilegios mínimos <code>elden-ring-agent-sa</code> dedicada, ver nota abajo (TODO-05, completado 2026-09-06).

Nota operativa (deploy): la identidad que despliega (`gcloud run deploy --source .`) necesitó dos roles adicionales no incluidos en `roles/editor` pese a su amplitud — `roles/storage.objectViewer` (para que Cloud Build lea el código fuente subido) y `roles/secretmanager.secretAccessor` sobre el secreto específico. Esto es sobre la identidad que despliega, no sobre la que corre el servicio (ver nota siguiente).

Nota operativa (runtime, actualizado 2026-09-06): Cloud Run ya no corre con la service account por defecto del proyecto (`roles/editor`, deuda técnica aceptada al cierre del MVP — ver sección 10). Ahora usa una SA dedicada de privilegios mínimos, `elden-ring-agent-sa@ah-estefania-alozno.iam.gserviceaccount.com`, con solo `bigquery.dataViewer`, `bigquery.jobUser`, `bigquery.connectionUser`, `aiplatform.user` y `datastore.user` a nivel de proyecto, más `secretmanager.secretAccessor` restringido al secreto `jwt-secret` (no a nivel proyecto). Nota no obvia: `bigquery.connectionUser` fue necesario porque `ML.GENERATE_EMBEDDING` (tool RAG) pasa por la conexión de BigQuery `vertex_ai_connection` — sin ese rol el chat fallaba con 403 pese a que la SA ya tenía `dataViewer` / `jobUser`. Verificado end-to-end con la cuenta QA `qa-test-chatux` tras el cambio.

Redespliegues post-MVP (historial navegable, indicador de progreso, fixes de memoria y de sesiones vacías — ver sección 6): `gcloud run deploy ... --source .`, sin repetir `--set-env-vars` / `--set-secrets` — Cloud Run conserva la configuración de la revisión anterior cuando no se la pisa explícitamente. Revisiones activas resultantes: `elden-ring-agent-00005-xnq` y, tras el fix de sesiones vacías, `elden-ring-agent-00006-2wg` (actual). Ambas verificadas en vivo contra el servicio desplegado, no solo localmente.

9. Evaluación

Dos niveles obligatorios, ninguno reemplaza al otro: evaluación manual (humana, 1-5) y LLM-as-a-judge (Gemini, salida estructurada). 13 casos, cubriendo las 8 categorías mínimas del PRD (lore, retrieval multilingüe, recomendación, recomendación+memoria, memoria persistente, corrección, olvido, fuera de corpus).

Métrica	Promedio	Objetivo	
Groundedness	4.85	≥ 4.0	cumple
Answer relevance	5.00	≥ 4.0	cumple
Personalization (donde aplica)	4.83	≥ 4.0	cumple
Fugas de memoria entre usuarios	0	0	cumple
Passwords en texto plano	0	0	cumple
Respuestas RAG con fuentes cuando corresponde	100%	100%	cumple

Metodología (cómo se calcularon estas cifras)

Los 13 casos se ejecutaron **una vez cada uno** contra el sistema real (Firestore + BigQuery + Vertex AI/Gemini reales, el mismo camino de código que `/chat` en producción) vía `evaluation/llm_judge.py`, con un usuario sintético (`eval-user-fase10`) para no ensuciar los perfiles reales. No son repeticiones promediadas por caso — cada promedio de la tabla anterior es sobre las 13 corridas (o sobre el subconjunto donde la métrica aplica: personalization solo promedia los 6 casos de recomendación/memoria). El juez automático (Gemini, salida estructurada) evalúa el turno final de cada caso; la evaluación manual es una revisión humana independiente de los transcripts completos, no solo del score del juez — ambas están documentadas en detalle en `evaluation/manual_evaluation.md`.

Evidencia de pruebas (autosuficiente, sin abrir archivos externos)

Prueba	Entrada	Comportamiento esperado	Resultado observado
RAG con fuentes	Tras declarar preferencia por katanas: "Given what you know about me, what weapon should I use?"	Recupera evidencia real vía <code>search_elden_ring_knowledge</code> y cita fuentes verificables, no genéricas	Recomienda 3 katanas con razones ligadas al perfil. **Sources** - weapon · Nagakiba - weapon · Meteoric Ore Blade - weapon · Uchigatana
Memoria entre sesiones	Login 1: "I main a bleed build with high arcane scaling." → logout (sesión ADK se reinicia) → Login 2: "What do you remember about my preferences?"	Recupera desde Firestore (<code>player_profiles</code>), no desde contexto conversacional — el contexto de sesión no sobrevive al logout	"I remember that your preferred build type is 'bleed' and your primary stat is 'arcane'." — verificado directamente contra el documento en Firestore.
Aislamiento entre usuarios	Usuario sintético A declara preferencias; usuario sintético B (<code>eval-isolation-b</code>) pregunta qué recuerda de él	B nunca ve las preferencias guardadas por A — 0 fugas de memoria	Confirmado: el perfil de B es independiente y no contiene datos de A. Ejecutado en vivo en esta sesión de evaluación (ver <code>manual_evaluation.md</code> , punto 5).

Transcripts completos de las dos primeras pruebas: casos `recommendation-memory` y `persistent-memory-preference` en `evaluation/transcripts.json`. No se incluyen capturas de navegador del servicio desplegado porque el entorno de esta sesión no tiene chromium/playwright instalado (ver Limitaciones) — la evidencia de arriba corre contra la misma infraestructura

real (Firestore/BigQuery/Vertex AI) y el mismo código que atiende `/chat` en Cloud Run, solo que invocada directamente en vez de vía navegador.

Hallazgo honesto, no maquillado: el juez automático marcó 1 de 13 casos (`lore-01`, sobre Malenia) como alucinación. La revisión manual encontró que es un **falso positivo del juez**: `semantic_documents` en BigQuery contiene dos entidades `boss` duplicadas para Malenia con listas de "drops" inconsistentes entre sí (verificado con una query directa a BigQuery). El agente reportó fielmente ambos registros y marcó la discrepancia explícitamente ("another record indicates...") — no inventó nada. Es un defecto de calidad de datos upstream, en la capa Gold de solo lectura, no del agente ni del RAG. Se documenta en detalle en `evaluation/manual_evaluation.md`.

Evidencia completa: `evaluation/cases.yaml` (casos), `evaluation/transcripts.json` (conversaciones completas reales), `evaluation/llm_judge_results.json` (veredictos estructurados por caso), `evaluation/manual_evaluation.md` (tabla de 8 métricas por caso + verificación explícita de los 6 criterios mínimos del PRD §36).

10. Decisiones y trade-offs

Vector Search (BigQuery) vs FAISS

Se usa `VECTOR_SEARCH` nativo de BigQuery sin índice ANN. Con ~1,208 documentos el escaneo exacto es rápido y evita mantener infraestructura de vectores separada (FAISS, un servicio de índice, sincronización) — trade-off correcto para este volumen; no se justificaría para un corpus de cientos de miles de filas.

Firestore vs BigQuery para memoria

Firestore para lecturas/escrituras frecuentes de bajo volumen por documento (perfil, mensajes) — BigQuery está optimizado para analítica sobre grandes volúmenes, no para el patrón de acceso "leer/escribir un documento por request" que necesita la memoria conversacional.

Un solo agente vs multiagente

Ver sección 5 — sin justificación funcional para orquestación entre agentes en este alcance.

IAM: de `roles/editor` a privilegios mínimos

El MVP priorizó velocidad de entrega usando la service account por defecto de Cloud Run (`roles/editor`) en vez de una identidad dedicada de mínimo privilegio — deuda técnica aceptada explícitamente (TODO-05). **Resuelta el 2026-09-06:** Cloud Run corre ahora con `elden-ring-agent-sa`, con solo los 5 roles de proyecto que el servicio necesita en tiempo de ejecución (ver sección 8 y tabla de sección 12).

Apps Script vs frontend dedicado

La decisión más significativa que se desvió del plan original. El PRD proponía Apps Script embebido en Google Sites; se implementó completo (Login/Chat, JWT, CORS) y se publicó como Web App, con `doGet()` registrando "Completada" sin error el 100% de las veces desde el editor y desde `curl`. Sin embargo el navegador real de la usuaria **nunca** pudo renderizarlo — error genérico de Google Drive, reproducible en múltiples cuentas, redes, dispositivos y navegadores, sin causa raíz diagnosticable del lado de Google. Ante un problema de infraestructura de terceros sin más palancas de control de nuestro lado, se reemplazó por un frontend estático (HTML/CSS/JS sin build) servido por el mismo FastAPI vía `StaticFiles` — mismo origen, sin CORS, sin Google Sites — patrón tomado de un proyecto hermano del curso (`ah-grupo-fundador`). Validado end-to-end en el navegador real de la usuaria tras el redeploy.

11. Limitaciones

- **Corpus comunitario:** los datos de `semantic_documents` provienen de fuentes wiki de la comunidad, no de un dataset curado — contiene al menos una inconsistencia real conocida (dos registros duplicados de Malenia con drops distintos, ver sección 9). No se corrige aquí porque la capa Gold es de solo lectura y pertenece al proyecto Medallion paralelo.

- **Grounding restringido a la KB:** el agente rechaza cualquier pregunta fuera del corpus recuperado, incluso si la respuesta "obvia" está en el conocimiento general del LLM — es una limitación deliberada (P-01), no un bug. Es una mitigación de prompt, no una validación técnica de backend (ver sección 4).
- **Persistencia de memoria probabilística:** depende de que el modelo decida llamar `update_player_memory`; no hay una validación estructurada de backend que compruebe la persistencia después del hecho (ver sección 6).
- **Autenticación simplificada:** sin email, verificación, recuperación de contraseña ni MFA. Registro público cerrado — alta de usuarios solo local (`scripts/create_user.py`).
- **Sesgo de idioma ocasional:** en una prueba anterior a la batería formal de evaluación se observó una respuesta en inglés con un carácter chino suelto. No se reprodujo en los 13 casos formales, pero el comportamiento multilingüe está mitigado, no garantizado al 100%.
- **Sin índice vectorial ANN:** aceptable para ~1,208 documentos; no escalaría sin cambios a corpus de órdenes de magnitud mayores.
- **CORS ya no aplica** (el frontend se sirve del mismo origen), pero quedaba como deuda documentada mientras existió el frontend Apps Script.

12. Mejoras futuras

TODO	Descripción
TODO-01	implementado Selector de historial navegable — ver sección 6, "Actualización post-MVP".
TODO-02	Feedback explícito del usuario (me sirvió / no me sirvió) sobre las respuestas.
TODO-03	Endpoints administrativos (gestión de usuarios, memoria, etc.).
TODO-04	Observabilidad avanzada (métricas, tracing, alertas).
TODO-05	implementado Hardening de IAM — SA dedicada de privilegios mínimos (<code>elden-ring-agent-sa</code>), completado 2026-09-06. Ver sección 8.
TODO-06	Restringir CORS a orígenes específicos (ya no aplica tras el cambio de frontend, pero documentado por si se reintroduce un cliente externo).
TODO-07	Índice vectorial ANN si el corpus crece órdenes de magnitud.
TODO-08	Frontend avanzado (más pantallas, mejor UX, posiblemente streaming NDJSON en <code>/chat</code>).

13. URL del servicio activo

`https://elden-ring-agent-704637212685.us-central1.run.app`

Frontend (Login/Chat) y API viven en la misma URL.

Cuentas de evaluación

Registro público cerrado (sección 7) — roster fijo de 3 cuentas. Las credenciales de evaluación se entregan al docente por un canal privado, separado de este PDF, y se verificó de antemano que la cuenta permite iniciar sesión y usar el flujo completo (chat, RAG, memoria) antes de la entrega.